Unit-1

- **WEB ESSENTIALS AND STYLE SHEETS**
- Clients, Servers, and Communication.
- The Internet - Basic Internet Protocols –
- The World WideWeb-HTTPrequestmessage-responsemessage-WebClients-WebServers-
- Markup Languages: XHTML. An Introduction to HTML – History – Versions -Basic XHTML Syntax and Semantics - Fundamentals of HTML.
- CSS-IntroductiontoCascadingStyleSheets–Features-CoreSyntax-StyleSheetsand HTML - Cascading and Inheritance - Text Properties – Positioning.

- **Client**
- A **client** is a computer/device or software that **requests services** from another computer (the server).
- Clients usually initiate the communication.
- Examples:
  - Web browser (Google Chrome, Firefox, Edge).
  - Mobile apps (Instagram, WhatsApp).
  - Email clients (Outlook, Gmail app).
- Clients are **service requesters**.

- **Server**
- A **server** is a computer/system that **provides services or resources** to clients.
- Servers are usually powerful machines with specialized software.
- Examples:
  - **Web Server** (Apache, Nginx) → serves web pages.
  - **Database Server** (MySQL, Oracle) → stores and provides data.
  - **Mail Server** → handles emails.
- Servers are **service providers**.

- **Client–Server Communication**
- Based on the **Client–Server model** of computing.
- **Steps in communication**:
  - Client sends a **request** (e.g., HTTP GET request).
  - Server **processes** the request.
  - Server sends a **response** (e.g., web page, data, or error message).
- Communication happens over a **network** (usually Internet or LAN).
- Protocols define **how communication happens**:
  - HTTP/HTTPS (for web browsing).
  - FTP (for file transfer).
  - SMTP/IMAP (for email).

- **Examples of Client–Server Communication**
- Typing www.wikipedia.org in a browser:
  - **Client (browser)** → sends HTTP request.
  - **Server (Wikipedia’s web server)** → sends back the web page.
- Sending a WhatsApp message:
  - **Client (your phone app)** → sends message to WhatsApp server.
  - **Server** → forwards it to recipient’s client app.

- **Examples of Client–Server Communication**
- Typing www.wikipedia.org in a browser:
  - **Client (browser)** → sends HTTP request.
  - **Server (Wikipedia’s web server)** → sends back the web page.
- Sending a WhatsApp message:
  - **Client (your phone app)** → sends message to WhatsApp server.
  - **Server** → forwards it to recipient’s client app.

- **Advantages of Client–Server Model**
- Centralized resources (data stored on servers).
- Easier maintenance (update server, all clients benefit).
- Scalability (many clients can connect).
- Security (controlled access to resources).

- **Client** = Requests services (browsers, apps).
- **Server** = Provides services (web, database, mail).
- **Communication** = Clients send requests → Servers process and respond.

The Internet - Basic Internet Protocols

- **The Internet**
- The **Internet** is a global network of interconnected computers that communicate with each other using standardized protocols.
- It allows sharing of information, communication, and access to resources.
- Services provided by the Internet:
  - **World Wide Web (WWW)** → Websites and browsing.
  - **Email** → Communication using mail servers.
  - **File Transfer** → Uploading/downloading files.
  - **Remote Access** → Accessing systems from anywhere.
- Internet as a **network of networks**.

- **What is a Protocol?**
- A **protocol** is a set of rules that defines **how computers communicate** over a network.
- Without protocols, devices would not understand each other.

- **. Basic Internet Protocols**

a) IP (Internet Protocol)

The backbone of the Internet.

Responsible for addressing and routing data from source to destination.

Each device has a unique IP address (IPv4: 192.168.1.1, IPv6: 2001:db8::1).

(b) TCP (Transmission Control Protocol)

Works with IP → Together called TCP/IP.

Provides reliable communication (ensures no data is lost).

Breaks data into packets, ensures they arrive in the correct order.

(c) UDP (User Datagram Protocol)

Faster, lightweight alternative to TCP.

Used when speed matters more than reliability.

Examples: Online gaming, video streaming, VoIP (calls).

(d) HTTP / HTTPS (Hypertext Transfer Protocol / Secure)

Used for web browsing.

HTTP → Transfers webpages between client (browser) and server.

HTTPS → Encrypted version (secure, uses SSL/TLS).

(e) FTP (File Transfer Protocol)

Used to upload and download files between client and server.

Example: Transferring website files to a hosting server.

(f) SMTP, POP3, IMAP (Email Protocols)

SMTP (Simple Mail Transfer Protocol) → Sending emails.

POP3 (Post Office Protocol v3) → Downloading emails (removes from server).

IMAP (Internet Message Access Protocol) → Accessing emails while keeping them on the server.

(f) SMTP, POP3, IMAP (Email Protocols)

SMTP (Simple Mail Transfer Protocol) → Sending emails.

POP3 (Post Office Protocol v3) → Downloading emails (removes from server).

IMAP (Internet Message Access Protocol) → Accessing emails while keeping them on the server.

(g) DNS (Domain Name System)

Converts domain names (like www.google.com) into IP addresses.

Works like a phonebook of the Internet.

- **Quick Example of Protocols Working Together**
- When you visit https://www.wikipedia.org:
- **DNS** translates domain name → IP address.
- **IP** locates the server.
- **TCP** establishes a reliable connection.
- **HTTPS** transfers the webpage securely.

- **World Wide Web (WWW)**

- The **World Wide Web (WWW)** is a system of interlinked hypertext documents accessible via the Internet.
- Uses **web browsers** (Chrome, Firefox, Edge, etc.) to view and navigate content.
- Documents are written in **HTML** (Hypertext Markup Language).
- Each resource on the web has a unique address called a **URL** (Uniform Resource Locator).
- Works on the **Client–Server model**.
- Example: When you type www.google.com, your browser (client) sends a request to Google’s server, which responds with the webpage.

- **HTTP (Hypertext Transfer Protocol)**
- HTTP is the **communication protocol** used between **web clients** and **web servers**.
- It is **stateless**, meaning each request is independent.
- Runs mostly over **TCP/IP** (Transmission Control Protocol / Internet Protocol).

- **HTTP Request Message**
- When a client (browser) wants data, it sends an HTTP Request.
- Structure of a Request:
- 1. Request Line → Method, URL, HTTP version
- Example: GET /index.html HTTP/1.1
- 2. Header Fields → Provide additional information (Host, User-Agent, Cookies, etc.)
- 3. Body (optional) → Data sent to server (used in POST, PUT methods).
- Common HTTP Methods:
- • GET → Request data (read only).
- • POST → Send data (e.g., form submission).
- • PUT → Update data.
- • DELETE → Remove data.

- **HTTP Response Message**
- After processing a request, the server sends back an HTTP Response.
- Structure of a Response:
- 1. Status Line → Protocol version, Status code, Reason
- o Example: HTTP/1.1 200 OK
- 2. Header Fields → Information about server, content type, length, etc.
- 3. Body → The actual content (HTML, image, video, JSON, etc.).
- Common Status Codes:
- 200 OK → Success.
- 301 Moved Permanently → Resource moved to a new URL.
- 404 Not Found → Resource not found.
- 500 Internal Server Error → Server issue.

- **. Web Clients**
- Software that **sends requests** to web servers and **displays responses**.
- Example: Web browsers (Chrome, Firefox, Safari, Edge).
- Functions:
  - Render HTML pages.
  - Execute scripts (JavaScript).
  - Store cookies, cache, and session data.

- **Web Servers**
- Software that **receives HTTP requests** from clients and **sends back responses**.
- Example: Apache, Nginx, Microsoft IIS.
- Functions:
  - Host and serve web content (HTML, CSS, JS, images, videos).
  - Handle client requests using HTTP/HTTPS.
  - Manage resources and security.

- **Client–Server Interaction Example**
- User enters http://example.com in browser.
- Browser (client) sends **HTTP GET request** to server.
- Server finds the requested file (index.html) and sends it back as **HTTP Response**.
- Browser receives response and displays the webpage

- **Introduction to Markup Languages**
- A **markup language** is a computer language used to **structure, describe, and format documents**.
- It uses **tags** (e.g., <p>, <h1>, <img>) to define how content should be displayed in a web browser.
- Examples: **HTML, XHTML, XML, Markdown**.

- **. HTML (Hypertext Markup Language)**
- HTML is the **standard language for creating web pages**.
- It defines the **structure of web documents** using a system of tags.
- A basic HTML document:
- <!DOCTYPE html>
- <html>
- <head>
- <title>My First Page</title>
- </head>
- <body>
- <h1>Hello, World!</h1>
- This is my first web page.
- </body>

- **. History of HTML**
- **1989–1991** → Tim Berners-Lee invented HTML as part of the World Wide Web project.
- **1993** → HTML 1.0 (first release).
- **1995** → HTML 2.0 (standardized by IETF).
- **1997–1999** → HTML 3.2 and HTML 4.0 introduced more features (tables, styles, scripts).
- **2000** → XHTML 1.0 (stricter, XML-based HTML).
- **2014 onwards** → HTML5 became the current standard (supports audio, video, canvas, semantic tags).

- **Versions of HTML**
- **HTML 1.0** → Basic text formatting, hyperlinks.
- **HTML 2.0** → Forms, images, tables.
- **HTML 3.2** → Support for scripting, applets, styles.
- **HTML 4.0/4.01** → Frames, CSS support, scripting improvements.
- **XHTML 1.0** → HTML reformulated as XML (stricter rules).
- **HTML5** → Multimedia support, semantic elements, mobile-friendly.

- **XHTML (Extensible Hypertext Markup Language)**
- XHTML = **HTML written as well-formed XML**.
- Developed by W3C to make HTML more consistent and strict.
- Benefits:
  - Cleaner, well-structured code.
  - Easy to integrate with XML-based technologies.
  - More reliable for browsers and mobile devices.

- Basic XHTML Syntax and Semantics
- XHTML follows stricter rules than HTML:
- **Syntax Rules in XHTML**
- 1. **Tags must be properly nested**
- 2. Text <!-- Correct -->
- 3. Text <!-- Wrong -->
- 4. **All tags must be closed**
- 5.
 <!-- Correct in XHTML -->
- 6.
 <!-- Wrong -->
- 7. **Lowercase for all tags and attributes**
- 8. Correct
- 9. <P>Wrong</P>
- 10. Attribute values must be in quotes
- 11.
- 12. **Root element <html> must include XML namespace**
- 13. <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
- 14. "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
- 15. <html xmlns="http://www.w3.org/1999/xhtml">
- 16. <head><title>Example</title></head>
- 17. <body> Hello XHTML! </body>
- 18. </html>

- **Semantics in XHTML**
- **Semantics** = meaning of tags.
- Tags should be used for their **intended purpose**:
  - <h1> … </h1> → Main heading.
  - … → Paragraph.
  - … → Emphasized text (instead of <i> for italics).
  - … → Important text (instead of <b> for bold).

- **HTML** = standard language of the web.
- **XHTML** = stricter, XML-based version of HTML.
- **HTML History** = evolved from simple text links → modern multimedia pages (HTML5).
- **XHTML Syntax** = lowercase tags, all attributes in quotes, all tags closed, proper nesting.
- **Semantics** = tags should carry meaning, not just appearance.

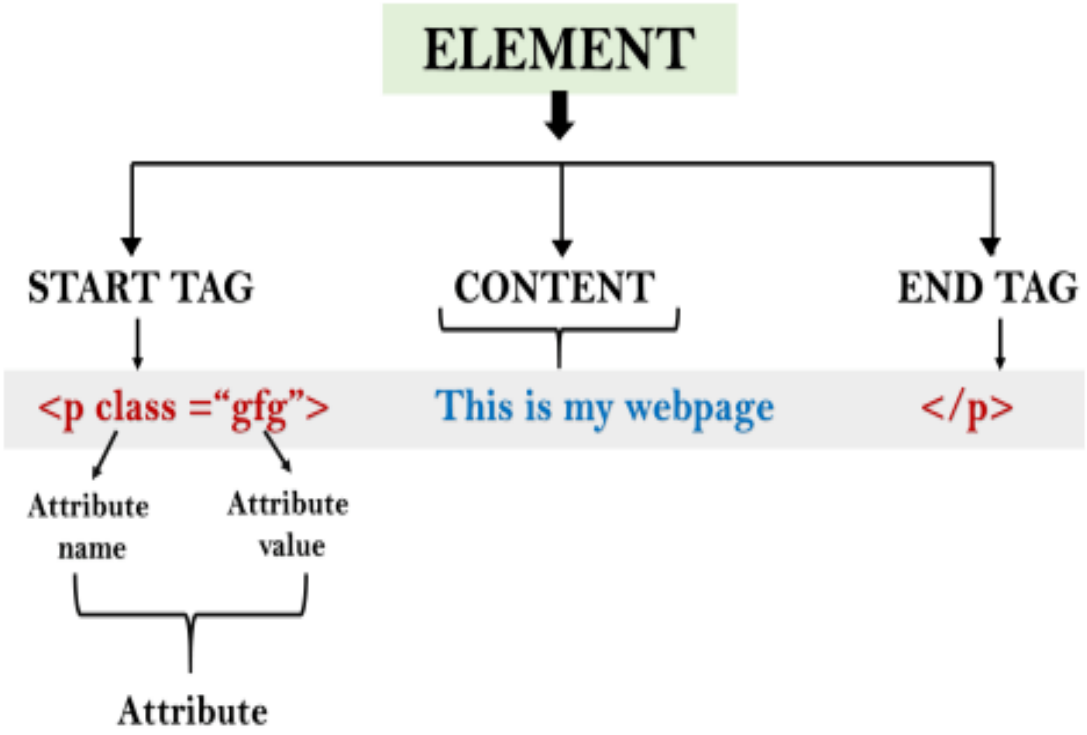
Feature	HTML	XHTML
Definition	Hypertext Markup Language – standard language for creating web pages.	Extensible Hypertext Markup Language – stricter, XML-based version of HTML.
Syntax rules	More flexible, browsers often “forgive” mistakes.	Very strict – must follow XML rules.
Tag Case	Tags can be uppercase or lowercase.	Tags must be lowercase only.
Closing Tags	Some tags may remain unclosed (e.g.,).	All tags must be closed properly (e.g.,).
Attribute Values	Quotes are optional (<input type=text>).	Quotes are mandatory (<input type="text" />).
Empty Elements	Can be written without / (e.g., <img>).	Must be written with / (e.g., <img />).
Error Handling	Browsers try to correct mistakes automatically.	No error tolerance – incorrect code may not display.
Document Structure	Doctype is optional.	Doctype declaration

Building blocks of HTML

- An HTML document consist of its basic building blocks which are:
- **Tags:** An HTML tag surrounds the content and apply meaning to it. It is written between < and > brackets.
- **Attribute:** An attribute in HTML provides extra information about the element, and it is applied within the start tag. An HTML attribute contains two fields: name & value.

Syntax

<tag name attribute_name= " attr _value"> content </ tag name>



- **Elements:** An HTML element is an individual component of an HTML file. In an HTML file, everything written within tags are termed as HTML elements.

HTML Tags

- HTML tags are like keywords which defines that how web browser will format and display the content. With the help of tags, a web browser can distinguish between an HTML content and a simple content. **HTML tags contain three main parts: opening tag, content and closing tag. But some HTML tags are unclosed tags.**
- When a web browser reads an HTML document, **browser reads it from top to bottom and left to right.** HTML tags are used to create HTML documents and render their properties. Each HTML tags have different properties.
- **An HTML file must have some essential tags so that web browser can differentiate between a simple text and HTML text**. You can use as many tags you want as per your code requirement.
- All HTML tags must enclosed within < > these brackets.
- Every tag in HTML perform different tasks.
- If you have used an open tag <tag>, then you must use a close tag </tag> (except some tags)

- Syntax
- **<tag> content </tag**
- *Note: HTML Tags are always written in lowercase letters. The basic HTML tags are given below*:

<!-- -->	This tag is used to apply comment in an HTML document.
<!DOCTYPE>	This tag is used to specify the version of HTML
<a>	It is termed as anchor tag and it creates a hyperlink or link.
<aside>	It defines content aside from main content. Mainly represented as sidebar.
<audio>	It is used to embed sound content in HTML document.

<b>	It is used to make a text bold.
<big>	This tag is used to make font size one level larger than its surrounding content. **(Not supported in HTML5)**
<body>	It is used to define the body section of an HTML document.
 	It is used to apply single line break.
<button>	It is used to represent a clickable button
<canvas>	It is used to provide a graphics space within a web document.
<caption>	It is used to define a caption for a table.
	It is used to align the content in center. **(Not supported in HTML5)**
<col>	It defines a column within a table which represent common properties of columns and used with the <colgroup> element.
<colgroup>	It is used to define group of columns in a table.

<dd>	It is used to provide definition/description of a term in description list.
<del>	It defines a text which has been deleted from the document.
	It defines a division or section within HTML
	It is used to emphasis the content applied within this element.
<embed>	It is used as embedded container for external file/application/media, etc.
<font>	It defines the font, size, color, and face for the content. **(Not supported in HTML5)**
<footer>	It defines the footer section of a webpage.
<form>	It is used to define an HTML form.
<frame>	It defines a particular area of webpage which can contain another HTML file. **(Not supported in HTML5)**
<frameset>	It defines group of Frames. **(Not supported in HTML5)**

<h1> to <h6>	It defines headings for an HTML document from level 1 to level 6.
<head>	It defines the head section of an HTML document.
<header>	It defines the header of a section or webpage.
<hr>	It is used to apply thematic break between paragraph-level elements.
<i>	It is used to represent a text in some different voice.
<iframe>	It defines an inline frame which can embed other content.
<img>	It is used to insert an image within an HTML document.
<input>	It defines an input field within an HTML form.
<u>	It is used to render enclosed text with an underline.
	It defines unordered list of items.

<label>	It defines a text label for the input field of form.
<legend>	It defines a caption for content of <fieldset>
	It is used to represent items in list.
<link>	It represents a relationship between current document and an external resource.
<mark>	It represents a highlighted text.
<marquee>	It is used to insert the scrolling text or an image either horizontally or vertically.
<nav>	It represents section of page to represent navigation links.
	It defines an ordered list of items.
<p>	It represents a paragraph in an HTML
<script>	It is used to declare the JavaScript within HTML document.
<small>	It is used to make text font one size smaller than documents base font size.

	It is used for styling and grouping inline.
<strike>	It is used to render strike through the text. **(Not supported in HTML5)**
	It is used to define important text.
<style>	It is used to contain style information for an HTML document.
<sub>	It defines a text which displays as a subscript text.
<summary>	It defines summary which can be used with <details> tag.
<sup>	It defines a text which represent as superscript text.
<var>	It defines variable name used in mathematical or programming context.
<video>	It is used to embed a video content with an HTML document

<table>	It is used to present data in tabular form or to create a table within HTML document.
<tbody>	It represents the body content of an HTML table and used along with <thead> and <tfoot>.
<td>	It is used to define cells of an HTML table which contains table data
<template>	It is used to contain the client side content which will not display at time of page load and may render later using JavaScript.
<textarea>	It is used to define multiple line input, such as comment, feedback, and review, etc.
<tfoot>	It defines the footer content of an HTML table.
<th>	It defines the head cell of an HTML table.
<time>	It is used to define data/time within an HTML document.
<title>	It defines the title or name of an HTML document.
<tr>	It defines the row cells in an HTML table

```
1. <!DOCTYPE>
2. <html>
3. <head>
4. <title>Web page title</title>
5. </head>
6. <body>
7. <h1>Write Your First Heading</h1>
8. <p>Write Your First Paragraph.</p>
9. </body>
10. </html>
```

Description of HTML Example

- **<!DOCTYPE>:** It defines the document type or it instruct the browser about the version of HTML.
- **<html** > :This tag informs the browser that it is an HTML document. Text between html tag describes the web document. It is a container for all other elements of HTML except <!DOCTYPE>
- **<head>:** It should be the first element inside the <html> element, which contains the metadata(information about the document). It must be closed before the body tag opens.
- **<title>:** As its name suggested, it is used to add title of that HTML page which appears at the top of the browser window. It must be placed inside the head tag and should close immediately. (Optional)
- **<body>** : Text between body tag describes the body content of the page that is visible to the end user. This tag contains the main content of the HTML document.
- **<h1>** : Text between <h1> tag describes the first level heading of the webpage.

- All the content written between body elements are visible on web page.
- **Void element:** All the elements in HTML do not require to have start tag and end tag, some elements does not have content and end tag such elements are known as Void elements or empty elements. **These elements are also called as unpaired tag**.
- **Some Void elements are
 (represents a line break) , <hr>(represents a horizontal line), etc.**
- **Nested HTML Elements:** HTML can be nested, which means an element can contain another element.

HTML Formatting

- **HTML Formatting** is a process of formatting text for better look and feel. HTML provides us ability to format text without using CSS. There are many formatting tags in HTML. These tags are used to make text bold, italicized, or underlined.

Bold Text

- <!DOCTYPE>
- <html>
- <body>
- Write Your First Paragraph in bold text.
- </body>
- </html>

Write Your First Paragraph in bold text.

Italic Text

- <!DOCTYPE>
- <html>
- <body>
- Write Your First Paragraph in italic text.
- </body>
- </html>

Write Your First Paragraph in italic text.

HTML Marked formatting

- <!DOCTYPE>
- <html>
- <body>
- <h2> I want to put a <mark> Mark</mark> on your face</h2>
- </body>
- </html>

I want to put a Mark on your face

Underlined and strike Text

- <!DOCTYPE>
- <html>
- <body>
- <u> <strike>Write Your First Paragraph with strikethrough</strike>.</u>

- </body>
- </html>

~~Write Your First Paragraph with strikethrough.~~

Superscript Text

- <!DOCTYPE>
- <html>
- <body>
- Hello ^{Write Your First Paragraph in superscript.}

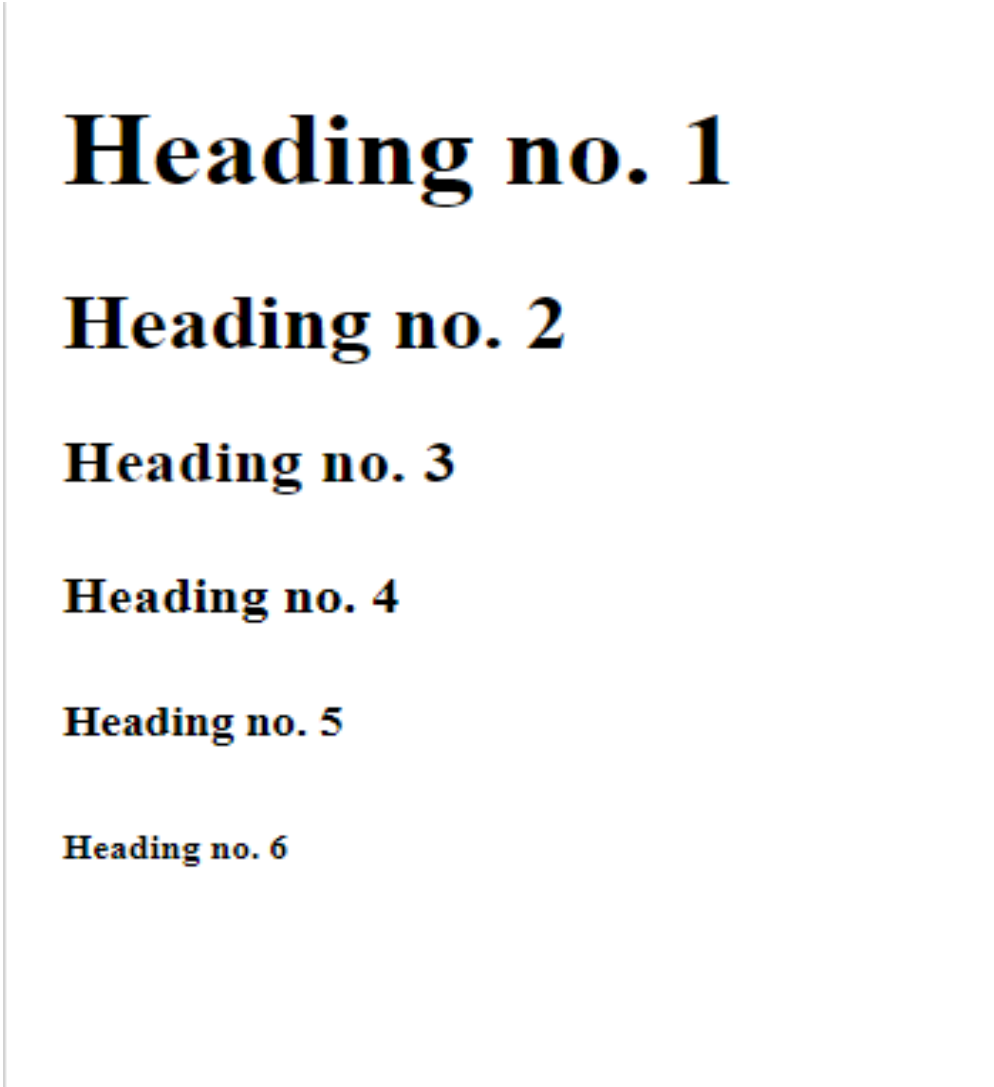
- </body>
- </html>

Hello Write Your First Paragraph in superscript.

HTML Heading

- A HTML heading or HTML h tag can be defined as a title or a subtitle which you want to display on the webpage. When you place the text within the heading tags <h1>.....</h1>, it is displayed on the browser in the bold format and size of the text depends on the number of heading.
- There are six different HTML headings which are defined with the <h1> to <h6> tags, from highest level h1 (main heading) to the least level h6 (least important heading).
- h1 is the largest heading tag and h6 is the smallest one. So h1 is used for most important heading and h6 is used for least important.

- <!DOCTYPE html>
- <html>
- <body>
- <h1>Heading no. 1</h1>
- <h2>Heading no. 2</h2>
- <h3>Heading no. 3</h3>
- <h4>Heading no. 4</h4>
- <h5>Heading no. 5</h5>
- <h6>Heading no. 6</h6>
- </body>
- </html>



Marquee HTML

- The **Marquee HTML** tag is a non-standard HTML element which is used to scroll a image or text horizontally or vertically.
- In simple words, you can say that it scrolls the image or text up, down, left or right automatically.
- Marquee tag was first introduced in early versions of Microsoft's Internet Explorer.

- <!DOCTYPE html>
- <html>
- <body>
- <marquee>This is an example of html marquee </marquee>
- </body>
- </html>

- <marquee width="60%" direction="up" height="100px">

  This is a sample scrolling text that has scrolls in the upper

  direction. </marquee>

HTML Hyperlinks (Links)

- A hyperlink (or link) is a word, group of words, or image that you can click on to jump to a new document or a new section within the current document.
- When you move the cursor over a link in a Web page, the arrow will turn into a little hand.
- Links are specified in HTML using the <a> tag.
- The <a> tag can be used in two ways:
  1)To create a link to another document, by using the href attribute
  2)To create a bookmark inside a document, by using the name attribute

HTML Link Syntax

- The HTML code for a link is simple. It looks like this:
  Link text
- The href attribute specifies the destination of a link.
  Eg: svnce

HTML Links - The target Attribute

- The target attribute specifies where to open the linked document.
- The example below will open the linked document in a new browser window or a new tab:

Eg: svnce

HTML Links - The name Attribute

- The name attribute specifies the name of an anchor.
- The name attribute is used to create a bookmark inside an HTML document.

HTML Images - The <img> Tag and the Src Attribute

- In HTML, images are defined with the <img> tag.
- The <img> tag is empty, which means that it contains attributes only, and has no closing tag.
- To display an image on a page, you need to use the src attribute. Src stands for "source".
- The value of the src attribute is the URL of the image you want to display.

Syntax for defining an image:

HTML Images - The Alt Attribute

- The required alt attribute specifies an alternate text for an image, if the image cannot be displayed.
- The value of the alt attribute is an author-defined text:

```
<img src="boat.gif" alt="Big Boat" />
```

- The alt attribute provides alternative information for an image if a user for some reason cannot view it

HTML Images - Set Height and Width of an Image

- The height and width attributes are used to specify the height and width of an image.
- The attribute values are specified in pixels by default:

```
<img src="pulpit.jpg" alt="Pulpit rock" width="304" height="228" />
```

HTML Tables

- Tables are defined with the <table> tag.
- A table is divided into rows (with the <tr> tag), and each row is divided into data cells (with the <td> tag). td stands for "table data," and holds the content of a data cell. A <td> tag can contain text, links, images, lists, forms, other tables, etc.

Table Example

```
<table border="1">
<tr>
<td>row 1, cell 1</td>
<td>row 1, cell 2</td>
</tr>
<tr>
<td>row 2, cell 1</td>
<td>row 2, cell 2</td>
</tr>
</table>
```

HTML Tables and the Border Attribute

- If you do not specify a border attribute, the table will be displayed without borders.
- Sometimes this can be useful, but most of the time, we want the borders to show.
- To display a table with borders, specify the border attribute:

```
<table border="1">
<tr>
<td>Row 1, cell 1</td>
<td>Row 1, cell 2</td>
</tr>
</table>
```

HTML Table Headers

- Header information in a table are defined with the <th> tag.
- All major browsers display the text in the <th> element as bold and centered.

```
<table border="1">
<tr>
<th>Header 1</th>
<th>Header 2</th>
</tr<>
<tr>
<td>row 1, cell 1</td>
<td>row 1, cell 2</td>
</tr>
<tr>
<td>row 2, cell 1</td>
<td>row 2, cell 2</td>
</tr>
</table>
```

Table with caption

```
<html>
<body>

<table border="1">
  <caption>Monthly
savings</caption>
  <tr>
    <th>Month</th>
    <th>Savings</th>
  </tr>
  <tr>
    <td>January</td>
    <td>$100</td>
  </tr>
  <tr>
    <td>February</td>
    <td>$50</td>
  </tr>
</table>

</body>
</html>
```

Colspan and rowspan

1)Colspan

```
<html>
<body>

<h4>Cell that spans two
columns:</h4>
<table border="1">
<tr>
  <th>Name</th>
  <th colspan="2">Telephone</th>
</tr>
<tr>
  <td>Bill Gates</td>
  <td>555 77 854</td>
  <td>555 77 855</td>
</tr>
</table>
```

2)rowspan

```
<h4>Cell that spans two
rows:</h4>
<table border="1">
<tr>
  <th>First Name:</th>
  <td>Bill Gates</td>
</tr>
<tr>
  <th
rowspan="2">Telephone:</th>
  <td>555 77 854</td>
</tr>
<tr>
  <td>555 77 855</td>
</tr>
</table>

</body>
</html>
```

Creating HTML Forms

The HTML <form> Elements

The HTML `<form>` element can contain one or more of the following form elements:

- `<input>`
- `<label>`
- `<select>`
- `<textarea>`
- `<button>`
- `<fieldset>`
- `<legend>`
- `<datalist>`
- `<output>`
- `<option>`
- `<optgroup>`

The <form> Element

- <form>
  .
  form elements
  .
  </form>

The <form> element is a container for different types of input elements,

such as: text fields, checkboxes, radio buttons, submit buttons, etc.

HTML <form> action Attribute

The `action` attribute specifies where to send the form-data when a form is submitted.

```
<form action="URL">
```

URL	Where to send the form-data when the form is submitted. Possible values: •An absolute URL - points to another web site (like action="http://www.example.com/example.htm") •A relative URL - points to a file within a web site (like action="example.htm")

The <input> Element

The HTML `<input>` element is the most used form element.

An `<input>` element can be displayed in many ways, depending on the `type` attribute.

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

Text Fields

The `<input type="text">` defines a single-line input field for text input

The <label> Element

The `<label>` tag defines a label for many form elements.

The `<label>` element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focus on the input element.

```
<html>
<body>

<h2>HTML Forms</h2>

<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>

<p>If you click the "Submit" button, the form-data will be sent to a page called "/action_page.php".</p>

</body>
</html>
```

Text input fields

First name:
John
Last name:
Doe

Note that the form itself is not visible.

Also note that the default width of text input fields is 20 characters.

Submitted Form Data

Your input was received as:

fname=John&lname=Doe

The server has processed your input and returned this answer.

Radio Buttons

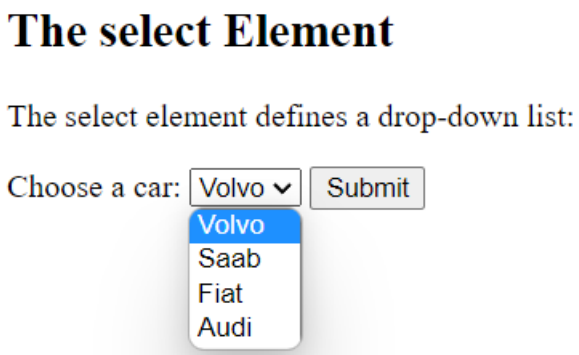
The `<input type="radio">` defines a radio button.

- <html>
- <body>
- <h2>Radio Buttons</h2>
- Choose your favorite Web language:
- <form>
- <input type="radio" id="html" name="fav_language" value="H
- <label for="html">HTML</label>
- <input type="radio" id="css" name="fav_language" value="CSS">
- <label for="css">CSS</label>
- <input type="radio" id="javascript" name="fav_language" value="JavaScript“ **checked**>
- <label for="javascript">JavaScript</label>
- </form>
- </body>
- </html>

Select element:

The `<select>` element defines a drop-down list:

```
<html>
<body>
<h2>The select Element</h2>
<p>The select element defines a drop-down list:</p>
<form action="/action_page.php">
  <label for="cars">Choose a car:</label>
  <select id="cars" name="cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
    <option value="fiat">Fiat</option>
    <option value="audi">Audi</option>
  </select>
  <input type="submit">
</form>
</body>
</html>
```

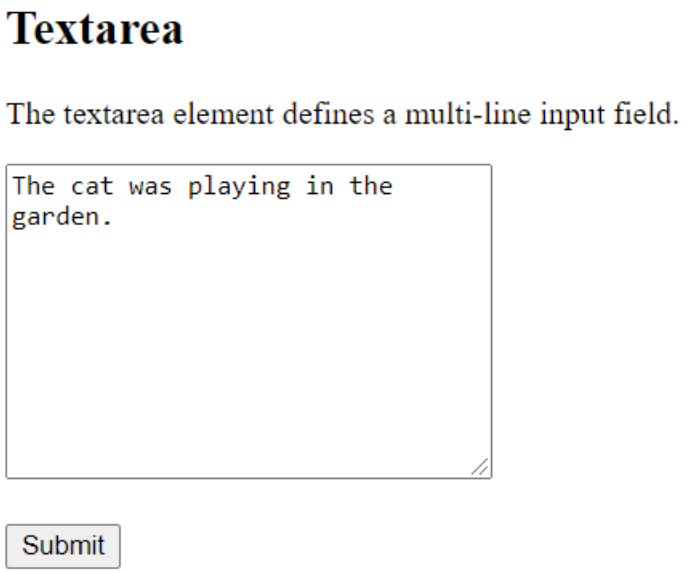


```
<html>
<body>

<h2>Textarea</h2>
<p>The textarea element defines a multi-line input field.</p>

<form action="/action_page.php">
  <textarea name="message" rows="10" cols="30">The cat was playing in the garden.</textarea>
  <br><br>
  <input type="submit">
</form>

</body>
</html>
```



```
<html>
<body>

<h2>The button Element</h2>

<button type="button" onclick="alert('Hello
World!')">Click Me!</button>

</body>
</html>
```

HTML Frames

- HTML frames are used to divide your browser window into multiple sections where each section can load a separate HTML document.
- A collection of frames in the browser window is known as a frameset.
- The window is divided into frames in a similar way the tables are organized: into rows and columns.

Disadvantages of Frames

- Some smaller devices cannot cope with frames often because their screen is not big enough to be divided up.
- Sometimes your page will be displayed differently on different computers due to different screen resolution.
- The browser's *back* button might not work as the user hopes.
- There are still few browsers that do not support frame technology.

Creating Frames

- To use frames on a page we use <frameset> tag instead of <body> tag.
- The <frameset> tag defines, how to divide the window into frames.
- The **rows** attribute of <frameset> tag defines horizontal frames.
- The **cols** attribute defines vertical frames.
- Each frame is indicated by <frame> tag and it defines which HTML document shall open into the frame.

If a user is using any old browser or any browser, which does not support frames then <noframes> element should be displayed to the user.
So you must place a <body> element inside the <noframes> element because the <frameset> element is supposed to replace the <body> element, but if a browser does not understand <frameset> element then it should understand what is inside the <body> element which is contained in a <noframes> element.

- <!DOCTYPE html>

```
<html>
<head>
<title>HTML Frames</title>
</head>
<frameset rows = "10%,80%,10%">     <frameset cols = "25%,50%,25%">
        <frame name = "top" src = "/html/top_frame.htm" />
        <frame name = "main" src = "/html/main_frame.htm" />
        <frame name = "bottom" src = "/html/bottom_frame.htm" /> <noframes>
        <body>Sorry!!Your browser does not support frames.</body>
</noframes>
</frameset>
</html>
```

The <frameset> Tag Attributes

Attribute	**Description**
Cols	Specifies how many columns are contained in the frameset and the size of each column. Four methods Absolute pixels ->cols = *"100, 500, 100“* Percentage of browser window -> *cols = "10%, 80%, 10%“* Using a wildcard symbol-> cols = *"10%, *, 10%“* Relative widths of the browser window -> *cols = "3*, 2*, 1*"*
rows	used to specify the number of rows in the frameset
border	This attribute specifies the width of the border of each frame in pixels. For example, border = "5". A value of zero means no border.
frameborder	This attribute specifies whether a three-dimensional border should be displayed between frames. This attribute takes value either 1 (yes) or 0 (no). For example frameborder = "0" specifies no border.
framespacing	This attribute specifies the amount of space between frames in a frameset. This can take any integer value. For example framespacing = "10" means there should be 10 pixels spacing between each frames.

The <frame> Tag Attributes

Attribute	Description
src	This attribute is used to give the file name that should be loaded in the frame. Its value can be any URL. src = "/html/top_frame.htm" will load an HTML file available in html directory.
name	This attribute allows you to give a name to a frame. It is used to indicate which frame a document should be loaded into. This is especially important when you want to create links in one frame that load pages into an another frame, in which case the second frame needs a name to identify itself as the target of the link.
frameborder	This attribute specifies whether or not the borders of that frame are shown; it overrides the value given in the frameborder attribute on the <frameset> tag if one is given, and this can take values either 1 (yes) or 0 (no).
marginwidth	This attribute allows you to specify the width of the space between the left and right of the frame's borders and the frame's content. The value is given in pixels. For example marginwidth = "10".
marginheight	This attribute allows you to specify the height of the space between the top and bottom of the frame's borders and its contents. The value is given in pixels. For example marginheight = "10".

The <frame> Tag Attributes contd….

Attribute	Description
noresize	By default, you can resize any frame by clicking and dragging on the borders of a frame. The noresize attribute prevents a user from being able to resize the frame. For example noresize = "noresize".
scrolling	This attribute controls the appearance of the scrollbars that appear on the frame. This takes values either "yes", "no" or "auto". For example scrolling = "no" means it should not have scroll bars.

CSS Id and Class

The id and class Selectors

- In addition to setting a style for a HTML element, CSS allows you to specify your own selectors called "id" and "class".

The id Selector

The id selector is used to specify a style for a single, unique element.

The id selector uses the id attribute of the HTML element, and is defined with a "#".

The style rule below will be applied to the element with id="para1":

```
<html>
<head>
<style type="text/css">
#para1
{
text-align:center;
color:red;
}
</style>
</head>

<body>
<p id="para1">Hello World!</p>
<p>This paragraph is not affected by the style.</p>
</body>
</html>
```

The class Selector

- The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements.
- This allows you to set a particular style for many HTML elements with the same class.
- The class selector uses the HTML class attribute, and is defined with a "."
- In the example below, all HTML elements with class="center" will be center-aligned:

```
<html>
<head>
<style type="text/css">
.center
{
text-align:center;
}
</style>
</head>

<body>
<h1 class="center">Center-aligned heading</h1>
<p class="center">Center-aligned paragraph.</p>
</body>
</html>
```

In the example below, all p elements with class="center" will be center-aligned:

```
<html>
<head>
<style type="text/css">
p.center
{
text-align:center;
}
</style>
</head>

<body>
<h1 class="center">This heading will not be affected</h1>
<p class="center">This paragraph will be center-aligned.</p>
</body>
</html>
```

- CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation of HTML documents. It controls the layout, colors, fonts, spacing, and overall visual appearance of web pages.

- Why Use CSS?
- Separates content (HTML) from presentation (CSS)

- Reusability across multiple pages

- Easier maintenance and cleaner HTML

- Enables responsive and visually appealing designs

- **Features of CSS**
- **Separation of content and style – HTML handles structure, CSS handles design.**
- **Improves presentation – Fonts, colors, spacing, layout, animations.**
- **Reusability – A single CSS file can style multiple web pages.**
- **Efficiency – Reduces code repetition.**
- **Responsive Design – Adjusts layout for different screen sizes.**
- **Global consistency – Same look and feel across a**

- **Core CSS Syntax**
- **CSS has a simple rule-based syntax:**
- **selector {**
- ** property: value;**
- **}**
- **Selector → Identifies which HTML element(s) to style.**
- **Property → The style feature to change (e.g., color, font-size).**
- **Value → The setting for the property.**
- **Example:**
- **p {**
- ** color: blue;**
- ** font-size: 16px;**
- **}**
- **This makes all <p> elements blue and sets font size to 16px.**

- Types of CSS
- 1. Inline CSS
- 2. Internal CSS (Embedded CSS)
- 3. External CSS

1. Inline CSS

- CSS is written directly in the HTML tag using the style attribute.

- Syntax:
- <h1 style="color: blue; font-size: 24px;">Hello World</h1>

- Advantages:
- Quick and easy for small changes
- Useful for testing/debugging

- Disadvantages:
- Not reusable
- Clutters HTML code

2. Internal CSS (Embedded CSS)

- CSS is written inside a <style> tag within the <head> of the HTML document.
- <!DOCTYPE html>
- <html>
- <head>
- <style>
- h1 {
- color: green;
- font-size: 30px;
- }
- </style>
- </head>
- <body>
- <h1>Welcome</h1>
- </body>
- </html>

- Advantages:
- Keeps styles in one place within the same file
- Good for small projects or single pages

- Disadvantages:
- Not reusable across multiple HTML files
- Makes HTML file heavier

3. External CSS

- CSS is written in a separate file with a .css extension and linked using the <link> tag in the <head> section.
- Syntax:
- HTML:
- <link rel="stylesheet" href="styles.css">
- styles.css:
- css
- h1 {
- color: red;
- font-size: 36px;
- }

- Advantages:
- Reusable across multiple HTML pages
- Keeps HTML clean
- Better maintainability for large projects

- Disadvantages:
- Requires additional HTTP request to load the stylesheet

Inline → Quick but messy.

Internal → Good for single-page styling.

External → Best practice for professional projects.

- **Cascading in CSS**
- The word *Cascading* means that when **multiple style rules** apply to the same element, the browser decides **which one takes priority**
- **Inline Styles** (highest).
- **Internal Styles**.
- **External Styles**.
- **Browser Defaults** (lowest).

- cell padding and cell spacing—two important concepts in HTML tables that affect the spacing inside and between table cells.
- 1. Cell Padding
- It is the space between the cell content and the cell border.
- It controls the inner spacing of the table cells.
- Example:
- Each cell will have **10px padding** inside (between content and border).<table border="1" cellpadding="10">
- <tr>
- <td>Cell 1</td>
- <td>Cell 2</td>
- </tr>
- </table>
- Each cell will have **10px padding** inside (between content and border).

- In CSS, you can also do this with:

- td {
- padding: 10px;
- }

- Cell Spacing
- It is the space between adjacent table cells.
- It controls the outer spacing between cells.

- Example:
- <table border="1" cellspacing="10">
- <tr>
- <td>Cell 1</td>
- <td>Cell 2</td>
- </tr>
- </table>
- There will be 10px space between the cells.

- In CSS, use border-spacing:

- table {
- border-spacing: 10px;
- }

Feature	Description	HTML Attribute	CSS Property
Cell Padding	Space **inside** the cell (content to border)	cellpadding="value"	padding (on td)
Cell Spacing	Space **between** the cells (border to border)	cellspacing="value"	border-spacing (on table)

- What is an HTML Form?
- An HTML form is used to collect user input and send it to a server for processing (e.g., login, registration, feedback).

- It uses the <form> tag, and inside it, we place form controls like text fields, checkboxes, radio buttons, and submit buttons.

- Basic Syntax
- <form action="submit_form.php" method="post">
- <!-- Form elements go here -->
- </form>

Attribute	Description
action	The URL where form data is sent for processing
method	HTTP method (GET or POST)
name	Name of the form
target	Where to display the response (_self, _blank, etc.)

Common Form Elements:

- 1. Text Input
- <label for="name">Name:</label>
- <input type="text" id="name" name="username">
- 2. Password Field
- <label for="password">Password:</label>
- <input type="password" id="password" name="pass">
- 3. Radio Buttons
- Gender:
- <input type="radio" id="male" name="gender" value="Male">
- <label for="male">Male</label>
- <input type="radio" id="female" name="gender" value="Female">
- <label for="female">Female</label>

- 4. Checkboxes
- Hobbies:
- <input type="checkbox" id="reading" name="hobby" value="Reading">
- <label for="reading">Reading</label>
- <input type="checkbox" id="traveling" name="hobby" value="Traveling">
- <label for="traveling">Traveling</label>

- 5. Dropdown (Select Menu)
- <label for="country">Country:</label>
- <select id="country" name="country">
- <option value="India">India</option>
- <option value="USA">USA</option>
- <option value="UK">UK</option>
- </select>

- 6. Textarea
- <label for="message">Message:</label>
- <textarea id="message" name="message" rows="4" cols="30"></textarea>
- 7. Submit Button
- <input type="submit" value="Submit">

- 8. Reset Button
- <input type="reset" value="Clear">
- Form Methods
- GET → Sends form data in the URL (used for search forms, not secure)

- POST → Sends form data in the request body (used for login, registration, secure data)

- <form action="process.php" method="post">
- <!-- form elements -->
- </form>

```
<!DOCTYPE html>
<html>
<body>
  <h2>Registration Form</h2>
  <form action="submit.php" method="post">
    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="firstname"><br><br>

    <label for="lname">Last Name:</label>
    <input type="text" id="lname" name="lastname"><br><br>

    <label>Gender:</label>
    <input type="radio" name="gender" value="Male"> Male
    <input type="radio" name="gender" value="Female"> Female<br><br>

    <label for="country">Country:</label>
    <select id="country" name="country">
      <option value="India">India</option>
      <option value="USA">USA</option>
    </select><br><br>

    <input type="submit" value="Register">
    <input type="reset" value="Clear">
  </form>
</body>
</html>
```

- Cascading in CSS
- Definition: CSS stands for Cascading Style Sheets. “Cascading” means styles are applied in order of importance and conflict resolution.
- Order of Precedence (from lowest to highest):
- Browser default styles
- External stylesheet
- Internal stylesheet (inside <style> tag in HTML)
- Inline styles (inside the element’s style attribute)
- !important declaration (highest priority)

- Hello
- Inline red will override external CSS p { color: blue; }.

- Inheritance in CSS
- Definition: Child elements can inherit property values from their parent elements.
- Inheritable Properties: font, color, line-height, visibility, text-align.
- Non-inheritable Properties: margin, padding, border, background.

-
- This text is green (inherited).
- This is also green.
-

- s — one with color: green and one without

- **Text Properties in CSS**
- Text properties control appearance and spacing of text.

- **Positioning in CSS**
- Defines how an element is placed on the web page.
- **Static (default):**
  Elements appear in normal document flow.
- p { position: static; }

- **Relative:**
  Positioned relative to its normal position.
- p { position: relative; top: 20px; left: 30px; }
- **Absolute**:
- Positioned relative to the nearest positioned ancestor (not the page).
- div { position: absolute; top: 50px; left: 50px; }
- **Fixed:**
  Stays fixed on screen even when scrolling.
- nav { position: fixed; top: 0; }

- **Sticky:**
  Acts like relative until scrolling reaches a threshold, then “sticks”.
- header { position: sticky; top: 0; }